EM1 — System Design Walkthroughs

"Walk me through how you'd design X" — EM1-specific answers only

How to use this doc:

- Monday: Read sections 1–3 (E2E system, human-in-the-loop, retrieval). Highest-risk. Know these cold.
- Tuesday AM: Sections 4–10 (technical depth on each subsystem).
- Frame every answer as: "Here's how I'd approach it for EM1, and here's what I built that's directly analogous."

---

1. FULL END-TO-END: THE VANTA-STYLE COMPLIANCE MODULE

"Walk me through how you'd design the Vanta-style intake and evaluation system." **Lead with this for any broad system design question. ~5 min, then let him steer.**

The problem to solve first (say this):

"The current flow is: plans arrive by email, someone manually uploads them, manually triggers the compliance check. The new module automates intake-to-results — the human re-enters only at the review step. That's the right place for them."

Stage 1 — Self-Serve Intake Form

Replaces email. Collects: plan type (EOP/HMP/COOP), agency (pre-populated from JWT), plan version/date, file upload, guideline library selection (FEMA standard or agency custom). File goes directly browser→S3 via **presigned PUT URL** — large PDFs never touch the API server.

On submit: `POST /evaluations` writes job record + publishes to SQS → returns `{job_id, status: "QUEUED"}` in < 200ms. The user sees a progress screen, not a spinner.

Double-submit protection: unique constraint on `(agency_id, plan_file_hash, guideline_set_id)`. Same plan re-submitted returns the existing `job_id` instead of starting a duplicate evaluation.

Stage 2 — Async Job Queue (why it must be async)

111 LLM calls × 1–3 seconds each = 2–5 minutes per evaluation. A synchronous HTTP response will time out. SQS → ECS worker is the only viable pattern. Worker updates `progress_counter` after each guideline; client polls `GET /evaluations/{job_id}` every 5 seconds for a live progress bar (`{completed: 45, total: 111}`).

Stage 3 — Document Ingestion (if new plan version)

Check `plan_file_hash` — if this exact file was already indexed, skip. Otherwise:

1. Structure-aware parsing: extract section map (section number → heading → page range) before any chunking. This is the load-bearing step for citation precision.
2. Two-level indexing: section summaries (~~500 tokens) for coarse navigation + paragraph chunks (~~150–200 tokens with overlap) for precision.
3. Every chunk tagged: `{agency_id, doc_id, section_number, heading, page, chunk_index}`.
4. Embed + write to pgvector; metadata to PostgreSQL.

Stage 4 — Crosswalk Evaluation Loop (111 guidelines)

Per guideline:

1. Hierarchical retrieval: query section summaries → top 2–3 sections → retrieve paragraph chunks within those sections only
2. LLM call with Pydantic schema: `{verdict: PASS|PARTIAL|FAIL|INCONCLUSIVE, evidence_quote: str, page_reference: str, gaps: list[str]}`
3. Evidence gate: if `evidence_quote` empty or not verbatim in retrieved context → force `INCONCLUSIVE`. No hallucination path to PASS.
4. Cache check first: if `(guideline_id, chunk_hash)` has a cached result → copy it, skip LLM call. Repeat evaluations are 60–70% cheaper.

Stage 5 — Human-in-the-Loop Review

The LLM verdict is first-pass, not final:

- **PASS (high confidence):** One-click accept
- **PARTIAL:** Evidence quote shown inline. EM accepts, overrides with annotation, or flags.
- **INCONCLUSIVE:** Mandatory review — cannot submit to FEMA without human resolution
- **FAIL:** Specific gaps listed. EM sees exactly what's missing.

Audit trail: LLM verdict + human decision stored together. Both go into the FEMA submission record.

Stage 6 — Report + History

Structured scorecard → PDF export for submission. History table: `{agency_id, plan_id, run_id, overall_score, run_date}` → enables trend view ("68% → 84% since last assessment").

---

2. HUMAN-IN-THE-LOOP — DESIGN PHILOSOPHY

"How do you think about where AI fits vs. where humans stay in the loop?"

The core distinction: Tasks where AI can be the final decision-maker vs. tasks where AI should accelerate a human decision. Crosswalk evaluation is firmly the second category.

Why: A FEMA compliance verdict requires contextual judgment the LLM doesn't have — the agency's local hazard profile, the jurisdiction's past assessment history, FEMA's current assessment priorities. The LLM has the document and the guideline text. That's it.

The right division of labor:

- LLM does: read 2000 pages, match 111 guidelines, extract verbatim evidence quotes, list specific gaps. Days of work in minutes.
- Human does: decide whether the evidence is sufficient for this specific agency's situation. That judgment is not automatable.

How the system enforces this division:

1. `INCONCLUSIVE` cannot be submitted — no bypass path
2. Evidence quotes are verbatim (verified) — human can spot-check in 5 seconds
3. LLM verdict + human decision both stored — full audit trail
4. Confidence scores surface low-confidence PASSes — system is honest about uncertainty, not just about failures

How it evolves: As the system accumulates history of LLM verdicts matching human decisions on stable plan sections, high-confidence PASSes on unchanged sections become one-click approvals. The loop gets shorter for routine cases. Human is still in the loop — just less friction over time.

---

3. RETRIEVAL FROM LARGE DOCUMENTS

"How do you handle retrieval when plans are 500–2000 pages?"

Three problems naive token chunking creates:

1. Context loss — a chunk has no idea it's from "Section 4.2: Resource Management"
2. Citation imprecision — "somewhere in the plan" is useless; "Page 91, Section 4.2" is what Smart References requires
3. Retrieval noise — top-5 chunks by pure vector similarity may span unrelated sections

Solution: structure-aware parsing + two-level indexing + hierarchical retrieval

Step 1 — Structure-aware parsing first. Before embedding anything, extract the section map: section number → heading → page range. For PDFs with inconsistent formatting, this is the hard engineering problem — look for numbered section headers, table of contents pages, heading font sizes. This metadata is the source of the citation — it is not extracted from LLM output.

Step 2 — Two-level indexing. Index section summaries (~~500 tokens, one per section) AND paragraph chunks (~~150–200 tokens, with overlap). Both in pgvector with the same embedding model.

Step 3 — Hierarchical retrieval per guideline. Query section summaries first → identify top 2–3 sections → retrieve paragraph chunks filtered to those sections only. This is the book-index analogy: find the chapter, then find the paragraph. Reduces noise, improves precision, reduces token cost.

Step 4 — Contextual chunk enrichment. Every chunk carries full provenance: `{doc_title, version, section_number, heading, page, paragraph_index}`. The LLM receives this alongside the text. The Smart References citation is assembled from the metadata — not generated by the LLM.

---

4. DECONFLICT ARCHITECTURE

> *"How would you design the cross-plan contradiction detection in E02?"*

The core problem: Two sections can be semantically *similar* but logically *contradictory*. "The EOC is activated by the Director" and "The Deputy Director activates the EOC" are near-identical embeddings that directly conflict. Vector similarity finds the similarity — it won't surface the contradiction.

Three-phase approach:

Phase 1 — Topic clustering across plans. Group sections by topic across all of an agency's active plans: "all sections from EOP, HMP, and COOP that address EOC activation." This creates candidate contradiction groups without comparing every section pair (which would be $O(n^2)$ across a large corpus).

Implementation: embed all sections, cluster by cosine similarity above a threshold (e.g., > 0.85), label each cluster with an extracted topic. Clusters with sections from >1 plan are contradiction candidates.

Phase 2 — Pairwise LLM contradiction check within clusters. For each candidate pair: "Given these two sections from different plans, do they give conflicting guidance on the same subject?" Structured output:

```
{contradicts: bool, topic: str, plan_a_section: {id, heading, excerpt},
plan_b_section: {id, heading, excerpt}, conflict_description: str}
```

Phase 3 — Conflict store. `{agency_id, topic, plan_a_id, plan_a_section_id, plan_b_id, plan_b_section_id, conflict_description, status: open|resolved, resolved_by, resolved_at}`. This is the data model for the Deconflict UI — open contradictions surfaced, resolved ones tracked.

Where a graph model helps: If contradiction volume grows, model sections as nodes and `contradicts` as edges. "All unresolved contradictions touching EOC activation across active plans" becomes a trivial graph traversal. Start with the PostgreSQL adjacency table; reach for Neo4j when the relational queries get complex. Same decision rule I applied at Samay.

---

5. AAR PIPELINE DESIGN

"How would you design the After Action Report ingestion and retrieval?"

What makes AARs different from plans (architecture-level):

- Unstructured narrative — no fixed section schema like a compliant EOP
- Incident-typed — a wildfire AAR is irrelevant to an active-shooter planner
- Lessons-forward — the valuable content is in "Lessons Learned" and "Recommendations" sections
- Cross-shareable (with consent) — other agencies' AARs are the most valuable content

Ingestion pipeline: Upload → S3 → structure-aware parsing (pattern-match on section markers: "Lessons Learned," "What Worked," "What Didn't Work," "Recommendations," "Timeline") → classify `incident_type` via LLM call or explicit selection at upload → chunk + embed → knowledge base.

Key metadata per AAR chunk:

```
{agency_id, doc_id, incident_type, section_type[lessons_learned|recommendations|
timeline|other], incident_date, is_shared, anonymize_agency}
```

Without `incident_type` tagging, retrieval is noisy across all incident types. This is the most important metadata field.

Two retrieval use cases:

1. E03 Chat: "What have other agencies learned from active shooter responses?" → filter `incident_type: active_shooter, section_type: lessons_learned` → semantic search within that set
2. Plan improvement: crosswalk flags PARTIAL on "EOC activation" → surface AARs from agencies with the same gap and how they resolved it

Stateful AAR authoring skill (LangGraph agent): After an incident, the agent guides the agency coordinator through writing the AAR step by step — structured timeline collection, what worked, what didn't, recommendations. At each step, retrieves relevant peer AARs as examples. State persists across sessions (coordinator can stop and resume). Final step: draft AAR generated from structured inputs. This is a stateful multi-step workflow agent, not a Q&A chatbot.

Cross-sharing governance: Explicit consent per document — `{is_shared: bool, anonymize_agency: bool, shared_incident_types: list}`. Agencies opt in per AAR. Anonymized sharing means other agencies see the content but not the source agency name.

---

6. AGENT SKILLS ARCHITECTURE (E03 DIRECTION)

"How would you design the agent skills layer in E03?"

The shift: From reactive stateless Q&A to stateful multi-step workflow agents. The agent isn't answering questions — it's guiding the user through a structured process with memory across turns and session restarts.

Four components:

State schema. Each skill defines a TypedDict state object that persists the full conversation state: current step, collected inputs, draft outputs, validation gaps. State lives in PostgreSQL, not in the context window. Example for plan creation skill:

```
class PlanCreationState(TypedDict):
    skill_id: str
    agency_id: str
    workflow_instance_id: str
    current_step: str            # "hazard_identification",
"mitigation_actions", etc.
    completed_steps: list[str]
    collected_inputs: dict       # structured data from each step
    draft_content: dict          # partial plan sections generated so far
    gaps: list[str]              # items flagged for human to fill
    status: Literal["ACTIVE", "PAUSED", "COMPLETE"]
```

Workflow graph. Each skill is a LangGraph directed graph. Nodes = steps. Edges = transitions. Branching based on tool results or user input. The AAR authoring skill: `[collect_incident_info] → [collect_timeline] → [collect_what_worked] → [collect_what_failed] → [collect_recommendations] → [retrieve_peer_examples] → [generate_draft]`.

Scoped tool access per node. Each node exposes only the tools relevant to its step. "Hazard identification" node: retrieve peer agency hazard sections from library, query FEMA doctrine for that hazard type. No write tools, no external API calls. Scoping keeps the agent focused and prevents unintended side effects.

Persistence and resumability. State keyed on `(agency_id, skill_type, workflow_instance_id)`. User can close the browser, come back tomorrow, pick up exactly where they left off. This is not optional for the plan creation skill — a plan takes days, not minutes.

Build order: Single-step skill, no branching, no persistence first. Prove the state schema + API contract + UI pattern end-to-end. Then add persistence. Then multi-step. Then branching. Complexity earned by demonstrated need.

---

7. ASYNC JOB ARCHITECTURE

> *"How would you handle the long-running evaluation pipeline?"*

Why async is mandatory: 111 LLM calls × 1–3s each = 2–5 minutes. A synchronous HTTP request will time out. The only correct answer is async with polling or webhook.

The full pattern:

```
POST /evaluations
  → validate input (plan exists, belongs to agency, guidelines selected)
```

```
  → write evaluation_jobs record (status: QUEUED)
  → publish to SQS
  → return {job_id, status: "QUEUED"} in < 200ms

SQS → ECS worker
  → update status: PROCESSING
  → for each guideline:
      check cache (guideline_id, chunk_hash)
      if cache hit: copy result, skip LLM
      if cache miss: hierarchical retrieval → LLM call → evidence gate → write
result
      increment progress_counter
  → update status: COMPLETED (or FAILED with error_message)

GET /evaluations/{job_id}
  → return {status, progress_counter, total_guidelines, results[]}
  → client polls every 5s → live progress bar
```

Schema:

```
CREATE TABLE evaluation_jobs (
  id UUID PRIMARY KEY,
  agency_id UUID NOT NULL REFERENCES agencies(id),
  plan_id UUID NOT NULL REFERENCES plans(id),
  guideline_set_id UUID NOT NULL,
  status TEXT NOT NULL DEFAULT 'QUEUED',   -- QUEUED|PROCESSING|COMPLETED|FAILED
  progress_counter INT NOT NULL DEFAULT 0,
  total_guidelines INT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  completed_at TIMESTAMPTZ,
  error_message TEXT,
  UNIQUE (agency_id, plan_file_hash, guideline_set_id)  -- idempotency
);

CREATE TABLE evaluation_results (
  id UUID PRIMARY KEY,
  job_id UUID NOT NULL REFERENCES evaluation_jobs(id),
  guideline_id UUID NOT NULL,
  verdict TEXT NOT NULL,   -- PASS|PARTIAL|FAIL|INCONCLUSIVE
  evidence_quote TEXT,
  page_reference TEXT,
  gaps TEXT[],
  cached_from_job_id UUID REFERENCES evaluation_jobs(id)
);
```

Idempotency: The unique constraint on `(agency_id, plan_file_hash, guideline_set_id)` means double-submitting the same plan returns the existing `job_id` — no duplicate evaluation kicked off.

Cache hit: Before each LLM call, check `evaluation_results WHERE guideline_id = $1 AND chunk_hash = $2`. Hit: copy result with `cached_from_job_id` reference. Plans are mostly stable between evaluations — caching eliminates 60–70% of LLM calls on re-runs.

---

8. LLM COST + SCALABILITY

"How does the system scale to 10x, and what does LLM cost look like?"

The math:

- Current: 111 guidelines × 60 agencies = 6,600 calls per full cycle
- At 10x agencies: ~66,000 calls/cycle
- Per-call cost (GPT-4o, ~1,000 output tokens): ~$0.005
- Per-evaluation cost: $0.55. Fine at any scale that matters near-term.
- The cost problem isn't per-evaluation price — it's re-running unchanged plan sections.

Three levers for cost efficiency:

Lever 1 — Result caching (biggest win, 60–70% reduction on re-runs). Key: `(guideline_id, chunk_hash)`. If a plan section hasn't changed since the last evaluation, the cached result is valid. Agencies update 10–15% of sections between evaluations — caching eliminates the rest. First evaluation is full price; every subsequent one is mostly cache hits.

Lever 2 — Model tiering (~35% reduction with no quality loss). Not all 111 guidelines need the full model. Structural checks ("Does the plan have a resource management section?") → GPT-4o-mini or Claude Haiku. Complex judgment calls ("Is this mitigation strategy adequately specific for the jurisdiction?") → full GPT-4o. Route at inference time based on guideline category. Rough split: 30–40% of guidelines are structural.

Lever 3 — Better retrieval = fewer tokens. Hierarchical retrieval delivers a smaller, more precise context window. Fewer tokens in = lower cost per call. Every improvement to retrieval quality has a direct cost payoff — it's not just an accuracy metric.

Scalability beyond LLM cost: SQS + ECS worker scales horizontally — add workers to reduce queue depth. pgvector with IVFFlat index handles millions of vectors. The bottleneck at scale is the evaluation worker throughput, which is addressable by adding worker replicas and tuning the SQS visibility timeout.

---

9. HALLUCINATION PREVENTION

> *"How do you ensure the LLM doesn't produce wrong verdicts that get submitted to FEMA?"*

The risk: LLM returns `PASS` with a plausible-sounding evidence quote that doesn't exist in the plan. Agency submits it. FEMA assessment fails. Trust in the platform is broken.

Four-layer defense:

Layer 1 — Evidence gating (the hard constraint). Pydantic schema requires `evidence_quote`. Post-processing step: verify the quote exists verbatim in the retrieved chunks. If not → force `INCONCLUSIVE`. This is not a soft heuristic — it is a hard gate. The system cannot emit a `PASS` without a verified, grounded quote. Every other anti-hallucination technique is defense-in-depth on top of this.

Layer 2 — Quote-then-reason prompting. Prompt structure: "First, copy the exact sentence from the provided context that addresses this guideline. Then, based only on that sentence, assign a verdict." Forces the LLM to locate evidence before reasoning. Eliminates the confabulation

pathway where the model reasons from training data instead of the retrieved context.

Layer 3 — Post-processing reconciliation. After LLM response: (a) verify evidence_quote exists verbatim in chunks, (b) verify page_reference exists in plan section metadata, (c) check for internal inconsistency — e.g., verdict is PASS but gaps list is non-empty. Flag inconsistencies for mandatory human review. Catches cases where the LLM is inconsistent with itself.

Layer 4 — Eval harness (drift detection). Golden test set: 20–30 hand-labeled evaluations (guideline + plan section + expert verdict). Run nightly. Track: precision/recall per guideline category, evidence verification pass rate, INCONCLUSIVE rate by guideline type. Not pass/fail — drift detection. Alert when precision drops > 5% from 7-day baseline. Catch quality degradation before a customer does.

---

10. MULTI-TENANT DESIGN

> *"60+ agencies — how do you ensure data isolation?"*

Three layers, each independent:

Layer 1 — Request-level. JWT contains `agency_id`. API middleware extracts it on every request, validates it against the session, injects it into request context. Every downstream call — database, vector store, S3 — receives `agency_id` explicitly. No request can touch data without an `agency_id` in its auth context.

Layer 2 — Database (PostgreSQL RLS). `agency_id` FK on every data table. Row-Level Security policy:

```
ALTER TABLE evaluation_jobs ENABLE ROW LEVEL SECURITY;
CREATE POLICY agency_isolation ON evaluation_jobs
  USING (agency_id = current_setting('app.current_agency_id')::UUID);
```

Even if application code has a bug and omits the filter, the database enforces it. Cross-tenant leakage is structurally prevented at the DB layer.

Layer 3 — Vector store. pgvector: `agency_id = $1` filter on every retrieval query. No cross-agency vector search is structurally possible. If using Pinecone: namespace per agency.

Guideline multi-tenancy: System guidelines have `agency_id IS NULL` (shared, read-only). Agency custom guidelines have `agency_id` set. Retrieval: `WHERE agency_id = $1 OR agency_id IS NULL`. Agencies cannot read each other's custom guidelines.

Document cross-sharing (Plan Library): Explicit consent model — agencies opt in per document with `{is_shared: bool, share_with_non_competing_agencies: bool}`. Shared documents are readable by other agencies but not modifiable. The `agency_id` of the document owner remains the authoritative tenant.

---

11. POSTGRESQL SCHEMA + PGVECTOR

> *"Walk me through the core data model."*

Core tables:

```
agencies(id UUID PK, name, state, created_at)

plans(id UUID PK, agency_id UUID FK, type TEXT,    -- EOP|HMP|COOP|IAP|AAR
  version TEXT, file_hash TEXT, s3_key TEXT, ingested_at TIMESTAMPTZ)

plan_sections(id UUID PK, plan_id UUID FK, section_number TEXT,
  heading TEXT, page_start INT, page_end INT)

plan_chunks(id UUID PK, plan_id UUID FK, section_id UUID FK,
  text TEXT, chunk_index INT, page INT,
  embedding vector(1536))                          -- pgvector column

guidelines(id UUID PK, agency_id UUID NULLABLE,   -- NULL = system/FEMA
  title TEXT, text TEXT, source TEXT, category TEXT)

evaluation_jobs(...)   -- see section 7
evaluation_results(...)  -- see section 7
```

Indexes:

```
-- ANN index for embedding retrieval
CREATE INDEX ON plan_chunks USING ivfflat (embedding vector_cosine_ops) WITH
(lists = 100);

-- Hierarchical retrieval: filter by plan then section before ANN
CREATE INDEX ON plan_chunks (plan_id, section_id);

-- Job status queries per agency (dashboard)
CREATE INDEX ON evaluation_jobs (agency_id, status);

-- Result aggregation per job
CREATE INDEX ON evaluation_results (job_id, guideline_id);
```

pgvector vs Pinecone decision:

- Use pgvector when: data is already in PostgreSQL, retrieval needs joins with structured metadata (filter by `plan_id` AND `section_id`), scale doesn't need horizontal vector scaling
- Use Pinecone when: pure semantic retrieval at millions of vectors, no join requirements, latency SLA < 10ms
- **For EM1: pgvector.** The hierarchical retrieval pattern (filter by section_id + ANN within result set) requires a join that would need a round-trip between Pinecone and PostgreSQL. One system, metadata joins for free.

---

12. TESTING + OBSERVABILITY FOR THE AI PIPELINE

> *"How do you keep the evaluation pipeline honest in production?"*

Deterministic tests (standard): API contract tests (Pydantic schema parses correctly), business logic (cache key logic, idempotency constraint, progress counter increment), integration (end-to-end: upload plan → job queued → worker runs → results written). These run in CI, fast, pass/fail.

AI behavior tests (different problem): LLM output is probabilistic. You can't assert exact content.

The approach:

1. Schema contract: does the response always contain `verdict`, `evidence_quote`, `page_reference`? Deterministic even though content varies.
2. Evidence gate regression: for golden examples with known evidence quotes, does the gate correctly flag quotes that are not verbatim in the context?
3. Golden test set: 20–30 labeled evaluations, run nightly, track precision/recall per guideline category — drift detection, not pass/fail.

Tracing (every LLM call): `run_id` = `{job_id}_{guideline_id}_{timestamp_hash}`. Log: retrieved chunk IDs + scores, prompt template version, model version, latency, input/output token counts, evidence gate result. When a user reports a wrong verdict on a specific guideline, replay the run — same context, same prompt, see what the model produces. Langfuse or LangSmith handles the storage and replay UI.

Production monitoring: | Metric | What a spike means | |---|---| | `INCONCLUSIVE` rate per guideline | Retrieval quality problem or guideline text changed | | Cache hit rate | Plans changing more than expected, or cache key logic broken | | Evidence gate failure rate | Leading indicator of hallucination pressure | | p99 evaluation latency | Slow workers or SQS queue backup | | Worker error rate | LLM API issues, Pydantic parse failures, network errors |

---

13. GRAPH-RAG POSITIONING

> *"Are you thinking about Graph-RAG for the retrieval layer?"*

Honest position: Graph-RAG is a specific solution to a specific problem — answering questions that require multi-hop relationship traversal across documents. For the main E01 crosswalk (one plan, 111 guidelines, single-document retrieval), it is overkill. Dense vector retrieval on well-structured chunks is faster, cheaper, and easier to debug.

Where Graph-RAG earns its cost for EM1:

1. Cross-agency queries: "Which other agencies in our region have mitigation strategies addressing our highest-risk hazards?" — cross-agency, cross-hazard, cross-section traversal. That's a graph query.
2. Deconflict cross-reference graph: sections as nodes, `contradicts` / `addresses_same_topic_as` as edges
3. Doctrine dependency chains: "this guideline requires that standard which requires this certification" — a graph by definition

Decision rule: If the critical queries are "find the most semantically similar content to this text" → vector (pgvector). If the critical queries are "traverse relationships between named entities across documents" → graph. If both → dual store with explicit routing.

What I'd propose: pgvector as the primary retrieval layer for all single-plan crosswalk queries. A lightweight adjacency model in PostgreSQL for Deconflict (sections and their contradiction relationships). Reach for Neo4j when the cross-document relationship traversals become complex

enough that SQL adjacency queries are painful — not before.

This is the same decision framework I applied at Samay: built the PostgreSQL version first, measured it, added Neo4j only when the query performance was genuinely unacceptable for the use case. Graph databases earn their operational cost; they don't justify it by default.